

Absolutely 👍 We'll pick up in the **next chat** with these together, because they fit nicely:

Next lesson 🧠 — Next.js Caching + Routers

1. Data Cache
2. Full Route Cache
3. Router Cache
4. How `revalidatePath()` / `revalidateTag()` interact with caching
5. App Router vs Pages Router

Show more ▼

Absolutely! 🤖 We'll continue **exactly** from here.

Lesson 5.7 — Next.js Caching + Routers 🧠🚀

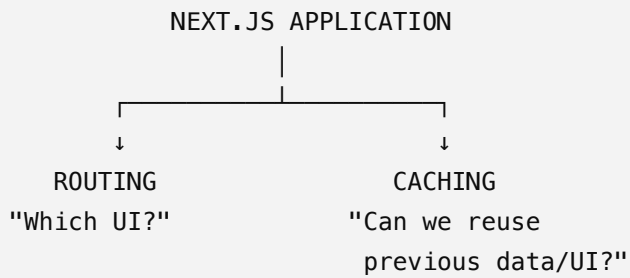
We'll go in this order:

1. Data Cache
2. Full Route Cache
3. Router Cache
4. `revalidatePath()` and `revalidateTag()`
5. How all 3 caches interact
6. What exactly is a Router?
7. App Router
8. Pages Router
9. Why Next.js has two routing systems
10. `app/` vs `pages/`
11. Server Components + Server Actions → App Router
12. Finally, connect **routing + rendering + caching + mutations** into one mental model.

And I'll keep doing it the way we've been doing the previous lessons: **one concept at a time, with concrete examples and stopping whenever something is confusing**. No jumping ahead. 👍

First mental picture

Before we touch any code, keep this distinction in mind:



Then later we'll see that Next.js actually has **three different caches**, which is where things get interesting:

```
graph TD; A[Data Cache] --> A1["Can I reuse this fetched data?"]; B[Full Route Cache] --> B1["Can I reuse the rendered result of this route?"]; C[Router Cache] --> C1["Can the browser reuse what it already received for this route?"]
```

That distinction is the key to making `revalidatePath()` and `revalidateTag()` finally make sense. 🔥

Let's start with #1: Data Cache.



ok

Lesson 5.7.1 — Data Cache (The easiest cache first) 🧠

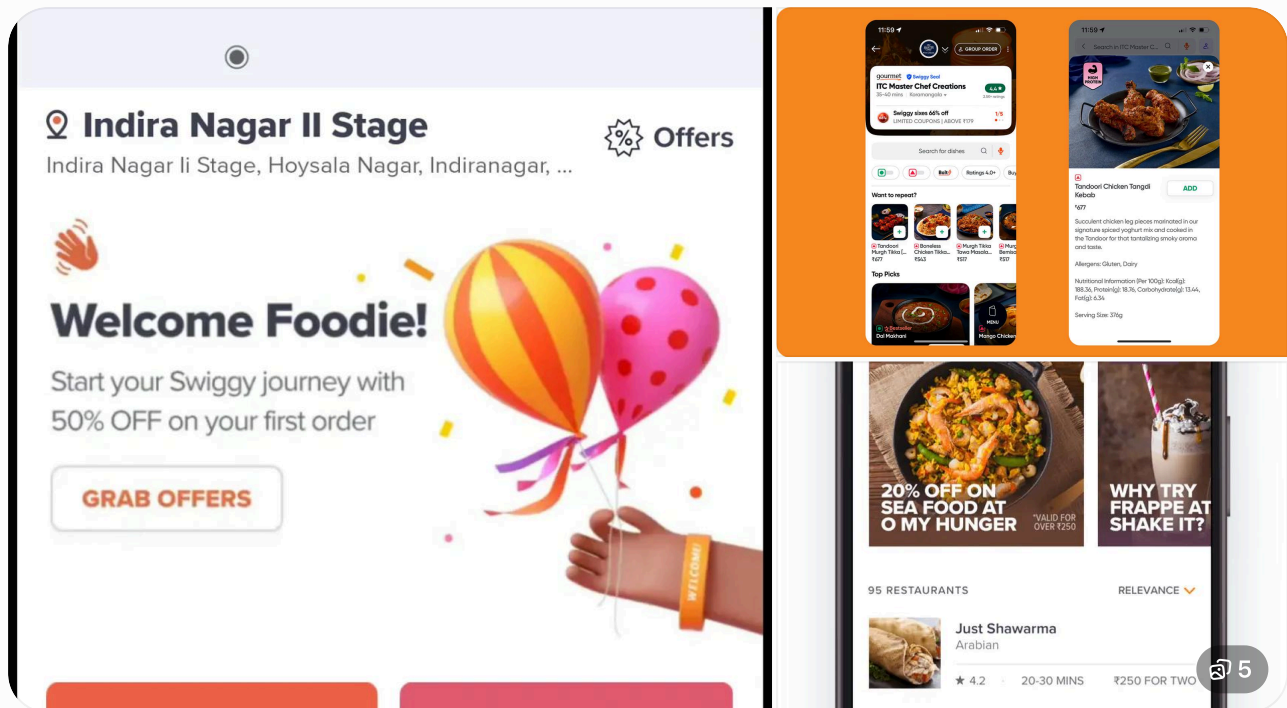
This is probably the **most important caching concept in Next.js**. If you understand this one properly, the other two become much easier.

As always... coffee ☕ ready? Let's go.

Step 1 — First, what is a cache?

Forget Next.js for one minute.

Imagine Swiggy.



You open Swiggy.

Swiggy asks its server:

"Give me all restaurants near Gopi."

Server sends 300 restaurants.

Now imagine you close and reopen the Home tab **2 seconds later**.

Should Swiggy ask the database again?

```
Open Home
|
Request restaurants
|
Database works
|
Returns restaurants
```



Then 2 seconds later...

```
Open Home Again
|
Request restaurants AGAIN?
```



That would be wasteful.

Instead Swiggy says:

"I already fetched this recently. I'll reuse it."

That reused copy is a **cache**.

Mental definition



Mental model

Cache = Saved result of previous work

Instead of repeating expensive work, we reuse an earlier result.

Step 2 — Where does Next.js cache?

This is where beginners get confused.

There are **three places** Next.js can cache.

Your Next.js Server

fetch(), Server Components, Server Actions

1. Data Cache

Saved fetch() responses

"Can I reuse API/database data?"

2. Full Route Cache

Saved rendered HTML/RSC output

"Can I reuse the whole page?"

3. Router Cache

Lives inside the browser

"Can navigation reuse downloaded routes?"

Browser

Router Cache lives here

Today we only learn the **blue box**.

Ignore the purple and orange ones for now.

Step 3 — What is Data Cache?

Imagine this page.

app/products/page.tsx

```
export default async function ProductsPage() {
  const products = await fetch(
    "https://api.store.com/products"
  ).then(res => res.json());

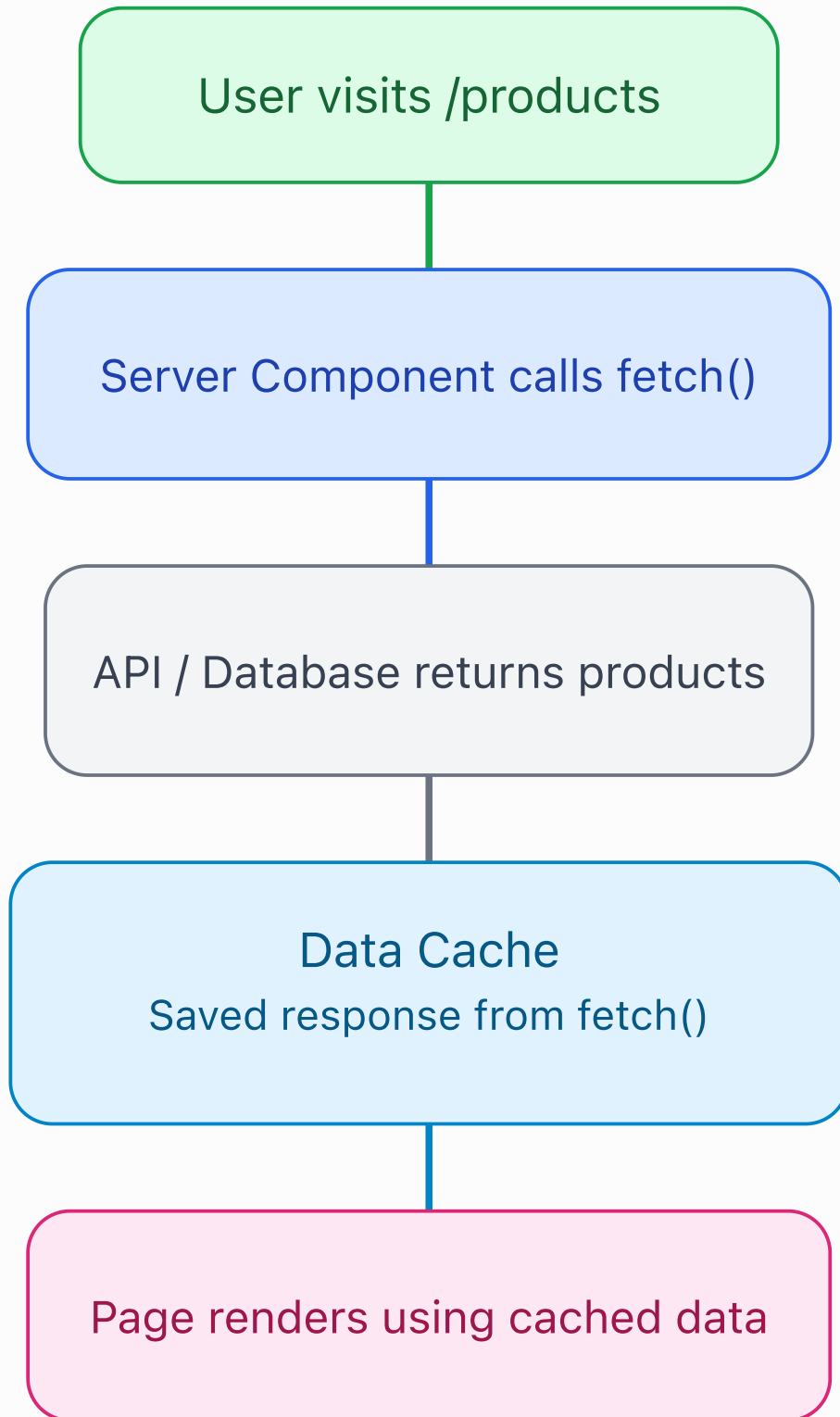
  return <ProductsList products={products} />;
}
```

Question:

When someone visits /products , what happens?

Step	What Next.js does
1	Server Component starts rendering.
2	<code>fetch()</code> requests products.
3	API/database returns products.
4	Next.js stores the response in the Data Cache.
5	Page renders using cached data.

Visual flow



The important sentence

Data Cache stores the result of `fetch()` on the Next.js server.

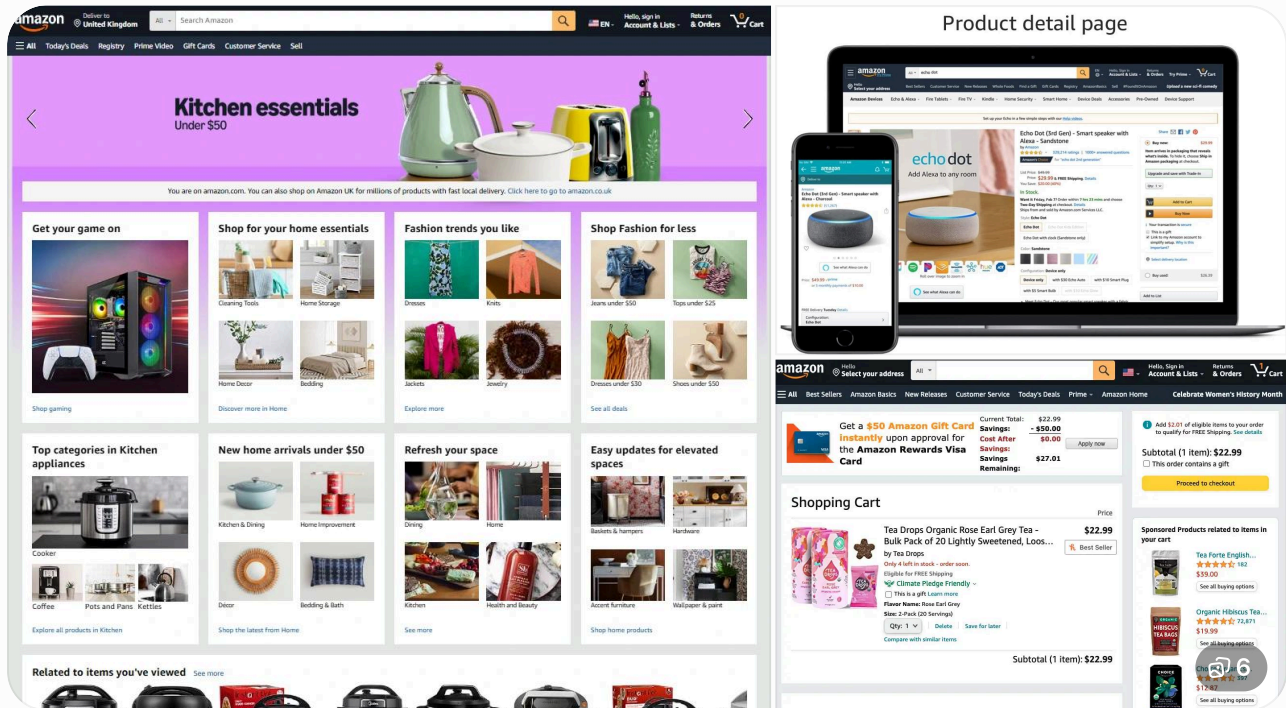
Not HTML.

Not React components.

Just the **data** returned by `fetch()`.

Step 4 — Let's build a shopping app

We'll use one app for the entire caching lesson.



Routes:

```
/products
/products/iphone
/cart
/profile
```

We'll revisit these routes in every lesson.

```
/products
```

Server fetches products.

```
const products = await fetch("/api/products")
```

Suppose API returns:

```
[
  { "id":1, "name":"iPhone" },
  { "id":2, "name":"MacBook" }
]
```

Next.js saves this exact JSON.



Data Cache now contains
GET /api/products

```
[
  { id:1, name:"iPhone" },
  { id:2, name:"MacBook" }
]
```

Think of it like a cupboard storing API responses.

Step 5 — Second visitor arrives

Now another person opens `/products` .

Question:

Does Next.js call API again?



No cache

Request API every visit.



Data Cache

Reuse previous response.

With Data Cache:

```
User 2
|
fetch("/api/products")
|
Next.js checks cache
|
Found!
|
Returns cached JSON
```



The API is **not contacted again**.

Huge performance improvement.

Step 6 — Wait... is fetch special in Next.js?

YES. Big YES. 🌟

This is one of the biggest differences between React and Next.js.

React

```
useEffect(() => {  
  fetch("/api/products")  
}, [])
```

Every browser fetches independently.

10 users = 10 fetches.

Next.js Server Component

```
const products = await fetch("/api/products")
```

Next.js automatically caches (by default for cacheable GET requests).

10 users can reuse the same cached response.

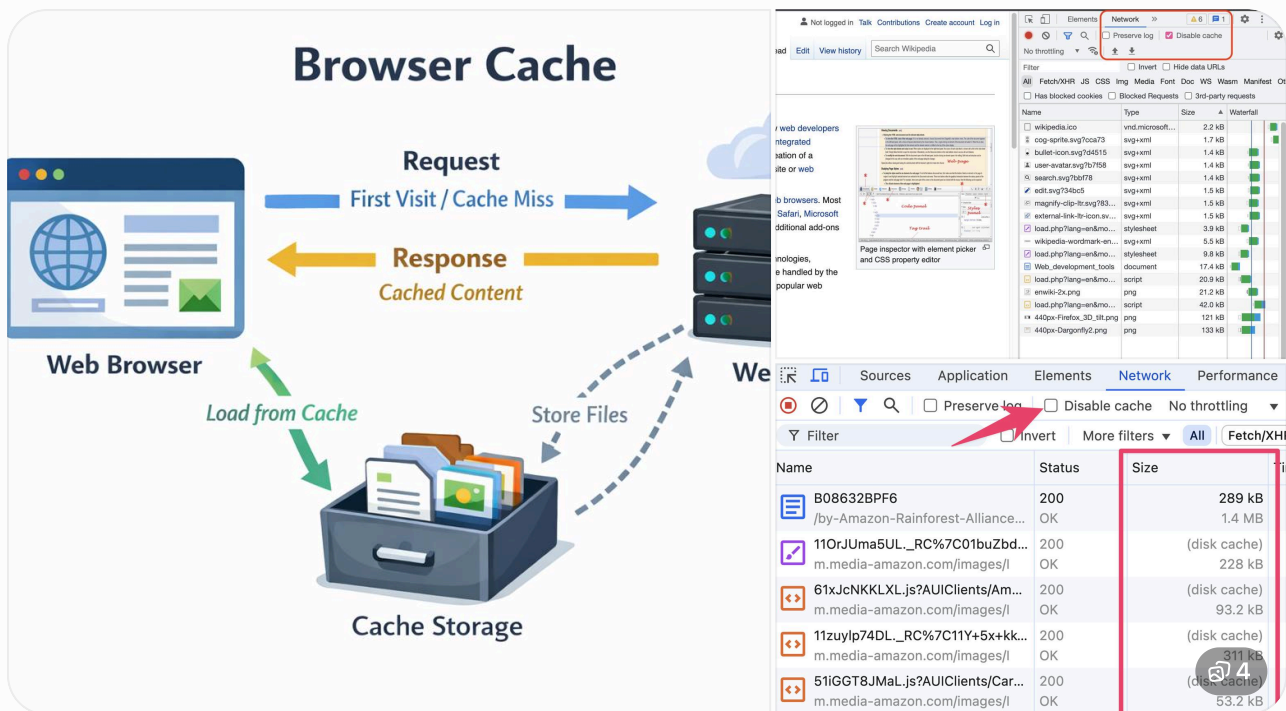
React	Next.js App Router
Browser fetches.	Server fetches.
Each user requests separately.	Server can reuse cached data.
No built-in fetch cache.	Built-in Data Cache.

This is why App Router feels "smart".

Step 7 — Does cache live in the browser?

This is where people mix Router Cache and Data Cache.

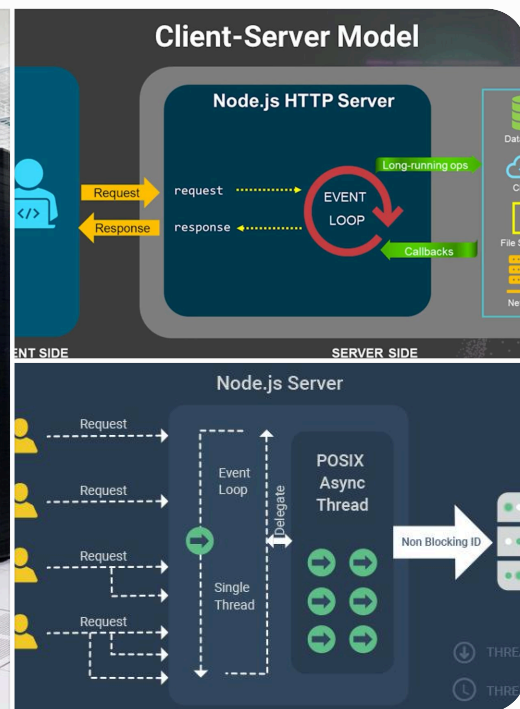
Browser cache



Lives on the user's device.

Different browser = different cache.

Data Cache



Lives on the **Next.js** server.

Shared across requests.



Mental picture

Data Cache belongs to the server.

Every user can benefit from it because the server already has the fetched response.

Step 8 — When is Data Cache useful?

Let's classify pages.

Page	Cache?
Product catalog	✓ Yes
Blog articles	✓ Yes
Countries list	✓ Yes
User notifications	✗ Usually no
Shopping cart	✗ Personalized
Live stock prices	✗ Too dynamic

Rule:

Cache things that **many users can reuse**.

Step 9 — Can we control the cache?

Absolutely.

Next.js lets you tell `fetch()` how to cache.

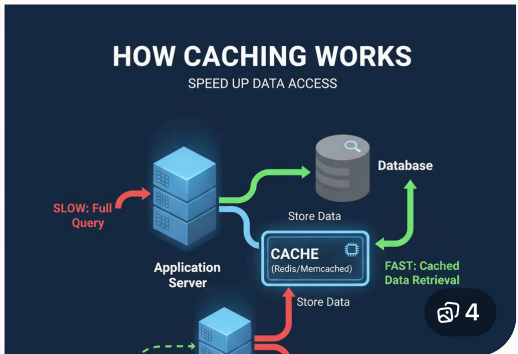
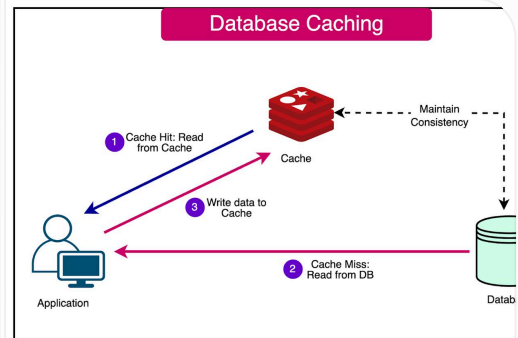
There are **three mental modes**.

Mode A — Force Cache (default for cacheable GETs)

```
await fetch("/api/products", {
  cache: "force-cache"
})
```

Meaning:

"Save this response and reuse it."



Use for:

- Products
- Blogs
- Categories
- Static CMS data

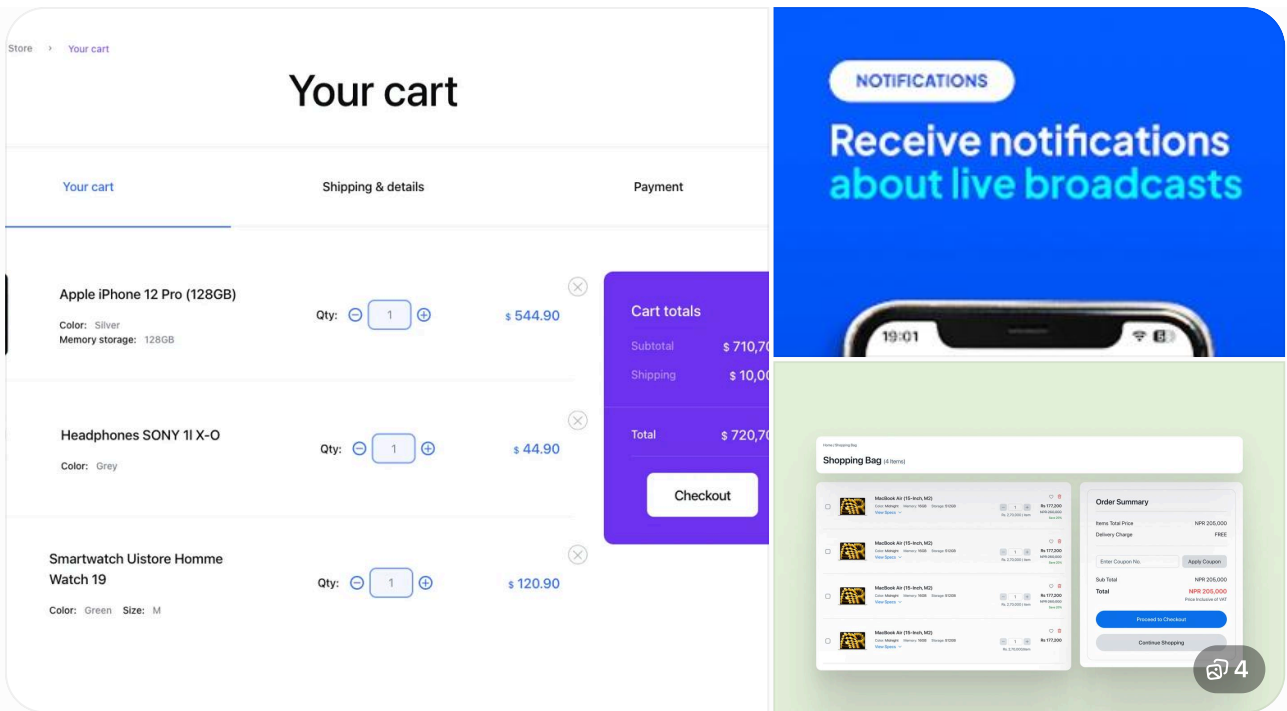
Mode B — No Store

```
await fetch("/api/cart", {
  cache: "no-store"
})
```

Meaning:

"Never cache this."

Every request goes to API/database.



Use for:

- Cart
- Notifications
- Dashboard
- User profile
- Live prices

Flow:

```
User
↓
fetch()
↓
API
↓
Database
```



Every single request.

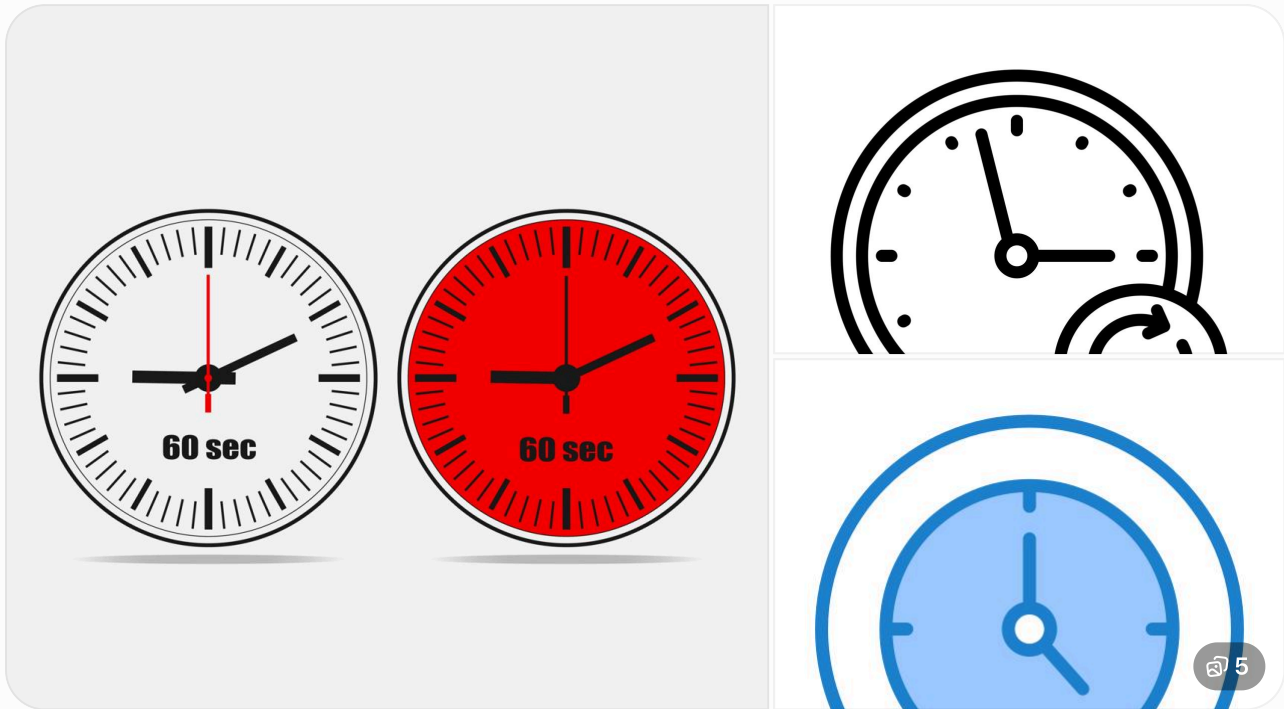
Mode C — Revalidate After Some Time

```
await fetch("/api/products", {
  next: { revalidate: 60 }
})
```

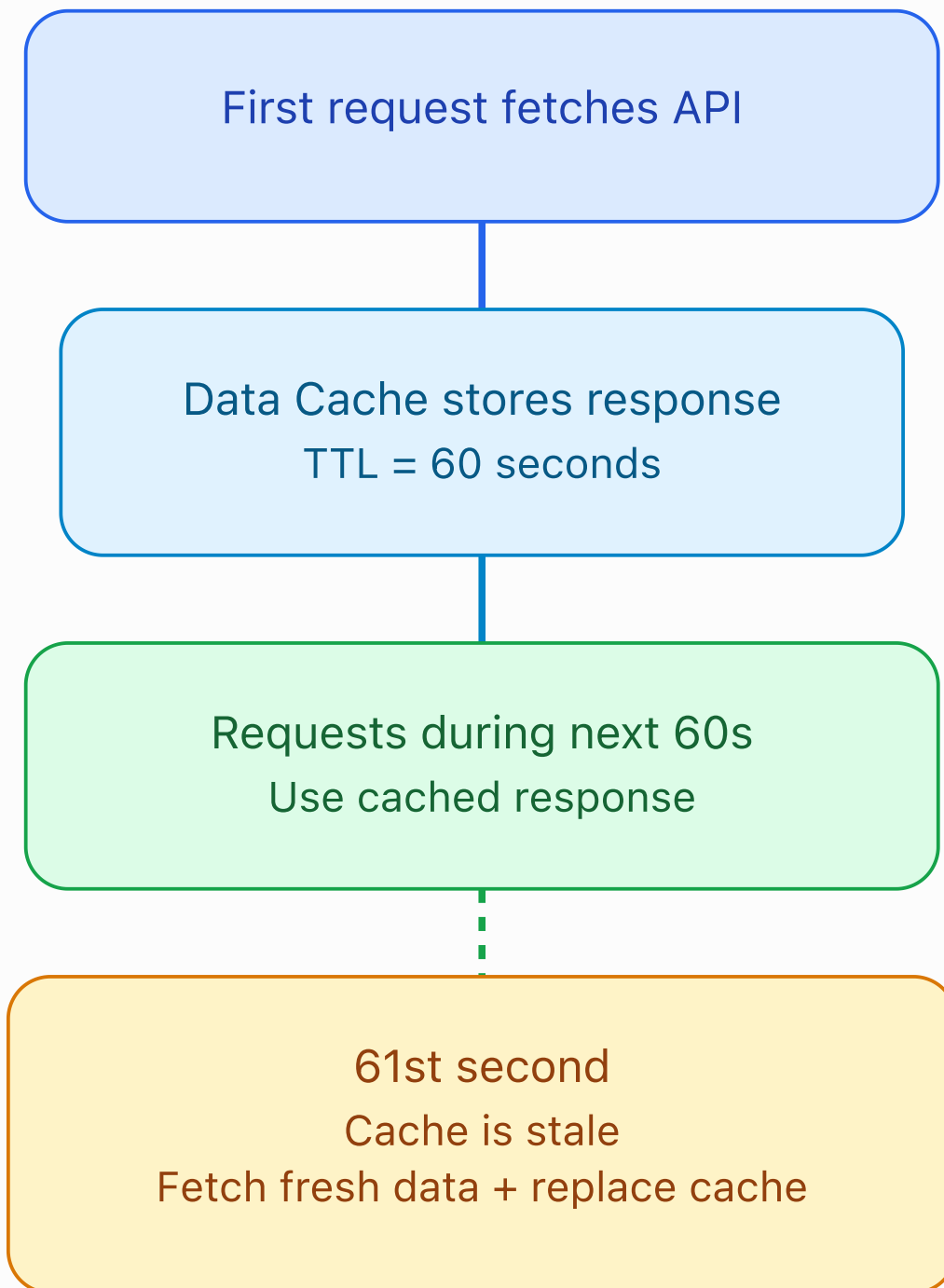
Meaning:

Cache for **60 seconds**.

This is called **Time-based Revalidation**.



Flow:



This is perfect for product prices that update every minute.

Step 10 — Real-world examples

Website	Best fetch option
Amazon products	revalidate: 60
Medium articles	force-cache

GitHub stars count	revalidate: 300
Twitter notifications	no-store
Bank balance	no-store

Notice how we choose caching based on **freshness needs**.

Step 11 — What if data changes before 60 seconds?

Imagine this.

The screenshot shows the Amazon Seller Central 'Manage Pricing' interface. A red arrow points to the 'Manage Pricing' link. The interface includes various pricing controls and a table of product listings.

Listing tools: All Inventory | Search suppressed and inactive listings (1)

Manage Pricing (highlighted with a red arrow)

FBA inventory tools: FBA opportunities | FBA Inventory

Manage Pricing Learn more

Action on 0 selected | Search SKU, Title, ISBN | Search | Filters: 2 applied

Filters:

Listing Status: ☐ All ☒ Active ☐ Inactive ☐ Incomplete ☐ Listing removed

☐ Save all ☐ Match Low Price

Product details: Image, SKU, Condition, Product Name, ASIN

Product 1: FDS-01 New

Product 2: Out of stock

Table Data:

Listing status	Product details	Performance	Inventory
Active	Product details	Sales: 112, Units sold: 1,338, Page views: 62,635	Available: 112, Inbound: 1,338, Reserve: 62,635
Out of stock	Product details	Sales: 66, Units sold: 1,650, Page views: 820,102	Available: 66, Inbound: 1,650, Reserve: 820,102

Current cache:

iPhone = ₹80,000

Seller updates price to ₹75,000.

But cache says:

Still ₹80,000

Oops.

This is called **stale cache**.

The cache is correct according to its timer, but the real data changed earlier.

So we need a way to say:

"Throw away this cached data immediately."

That is exactly what `revalidatePath()` and `revalidateTag()` do.

We'll learn them in Lesson **5.7.4**.

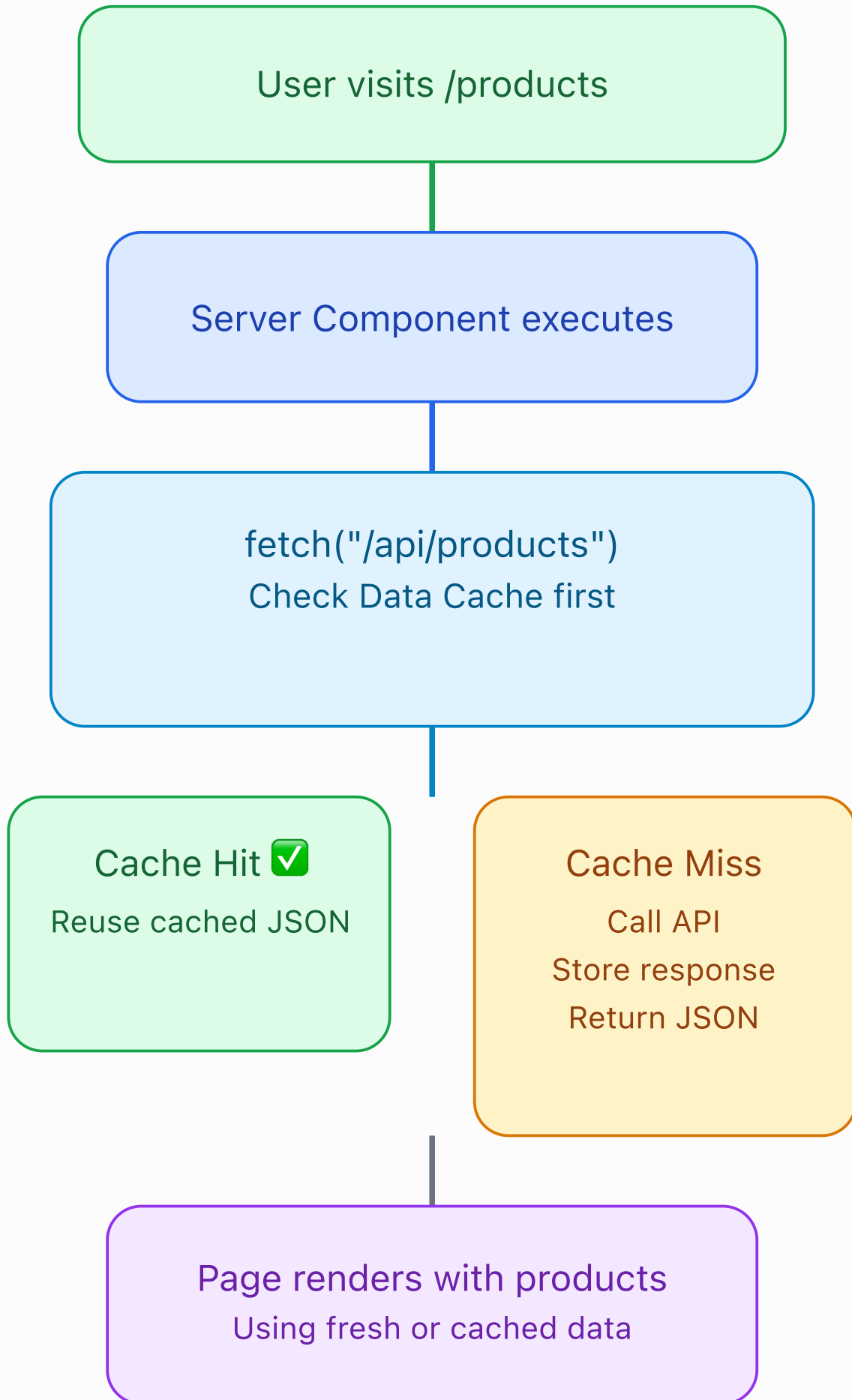
Step 12 — Important distinction: Data Cache vs React memoization

These sound similar but are different.

React memoization	Next.js Data Cache
<code>useMemo()</code>	<code>fetch()</code>
Lives during render.	Lives on server between requests.
Per user render.	Shared across requests.
Caches calculations.	Caches fetched data.

Very different purposes.

Step 13 — Data Cache in one complete flow





Lesson 5.7.1 Summary (Interview + Mental Model)

Data Cache — Cheat Sheet

Concept	Remember this
What is Data Cache?	Saved result of <code>fetch()</code> on the Next.js server.
What gets cached?	API/database response data (JSON, etc.).
Where does it live?	On the Next.js server, shared across requests.
Why use it?	Avoid repeated API/database calls and speed up rendering.
cache: "force-cache"	Reuse cached data whenever possible.
cache: "no-store"	Always fetch fresh data.
next: { revalidate: 60 }	Cache data for 60 seconds, then refresh.



Stop here (just like our course)

Don't jump to Full Route Cache yet.

Next: Lesson 5.7.2 — Full Route Cache

We'll answer a question that confuses almost everyone:

"If Data Cache already stores fetched data... why does Next.js need another cache called Full Route Cache?"

That lesson has the biggest "Aha!" moment of the caching chapter because you'll finally see the difference between **caching data** and **caching an entire rendered page**.



Sources

ChatGPT can make mistakes. Check important info.